GUÍA DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	INTRODUCCION AL DESARROLLO DE NUEVAS PLATAFORMAS (E)				
TÍTULO DE LA PRÁCTICA:	Listas filtrables con LazyColumn en Jetpack Compose				
NÚMERO DE PRÁCTICA:	8	**AÑO LECTIVO:**	2026B	**NRO. SEMESTRE:**	VIII
TIPO DE PRÁCTICA:	**INDIVIDUAL**				
	GRUPAL	**X**	**MÁXIMO DE ESTUDIANTES**		2
FECHA INICIO:	20/10/2026	**FECHA FIN:**	24/10/2026	**DURACIÓN:**	100 minutos
RECURSOS A UTILIZAR: Android Studio (Quail o superior), Kotlin, Jetpack Compose, emulador Android o dispositivo físico (API 34 o superior)					
DOCENTE(s): Roxana Evelyn Limache Calatayud					

OBJETIVOS/TEMAS Y COMPETENCIAS	
OBJETIVOS: - Comprender el Composable LazyColumn y su equivalencia con RecyclerView para mostrar listas de forma eficiente. - Implementar una lista de edificios con al menos tres datos por elemento (título, categoría y descripción) usando LazyColumn. - Aplicar un campo de búsqueda que filtre la lista en tiempo real conforme el usuario escribe.	
TEMAS: - LazyColumn y LazyRow en Jetpack Compose - items() y claves (key) para listas eficientes - Filtrado de listas en tiempo real con TextField y remember	
COMPETENCIAS	C.a Aplica de forma flexible técnicas, métodos, principios, normas, estándares y herramientas de ingeniería necesarias para la construcción de software e implementación de sistemas de información.
	C.b Investiga nuevos modelos, metodologías, técnicas, herramientas y tecnologías por ser necesarias para mantener la vigencia en el desempeño profesional.
	C.c
	C.d

CONTENIDO DE LA GUÍA

I. MARCO CONCEPTUAL

Revisar el siguiente recurso antes de la sesión:

- Video: "LazyColumn en Android Jetpack Compose. Lista con texto y botones"
https://www.youtube.com/watch?v=C4EkfVFC0cg

- Android Developers, listas en Compose:
https://developer.android.com/develop/ui/compose/lists

De RecyclerView a LazyColumn

En el enfoque clásico de Views, mostrar una lista larga y eficiente requería un RecyclerView en el XML, un Adapter que creara y vinculara los datos, y un ViewHolder por cada tipo de fila, reciclando vistas manualmente conforme el usuario hace scroll. En Jetpack Compose, LazyColumn cumple el mismo propósito con un solo Composable: solo compone (dibuja) las filas visibles en pantalla y las recicla automáticamente al hacer scroll, sin necesidad de Adapter ni ViewHolder, como se resume en la Figura 1.

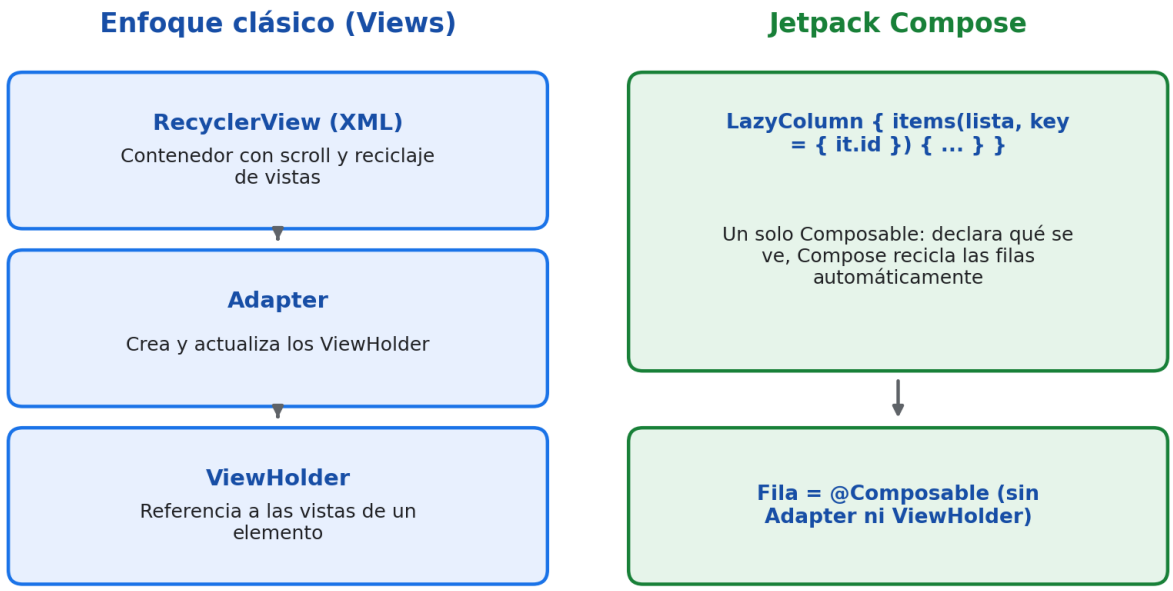


Figura 1. De RecyclerView + Adapter + ViewHolder a un único Composable LazyColumn (elaboración propia).

LazyColumn, items() y claves

Dentro de LazyColumn { ... } se usa la función items(lista) { elemento -> ... } para declarar cómo se dibuja cada fila a partir de una lista de datos. Es recomendable pasar también un key = { it.id } para que Compose identifique cada fila de forma estable entre recomposiciones, evitando recomposiciones innecesarias o pérdida de estado al filtrar o reordenar la lista.

Filtrado de listas en tiempo real

Para filtrar una lista mientras el usuario escribe, se guarda el texto de búsqueda en una variable de estado (var texto by remember { mutableStateOf("") }) ligada a un TextField, y en cada recomposición se calcula listaCompleta.filter { ... } usando ese texto como criterio. Como listaFiltrada depende de texto, cada vez que el usuario escribe un carácter, Compose recalcula el

filtro y recompone la LazyColumn mostrando solo las coincidencias, como se resume en la Figura 2.

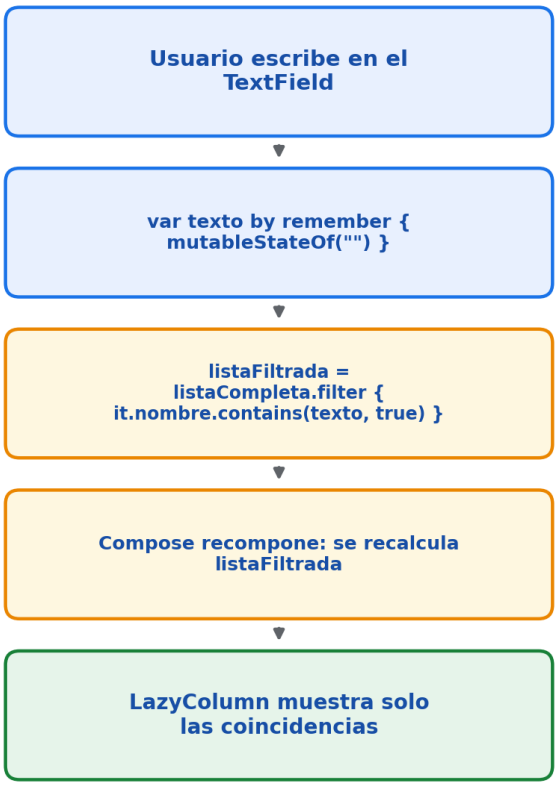


Figura 2. Flujo de filtrado de una lista en tiempo real conforme el usuario escribe (elaboración propia).

II. EJERCICIO/PROBLEMA RESUELTO POR EL DOCENTE

A continuación se detallan, paso a paso, las instrucciones para mostrar una lista de edificios del campus con un campo de búsqueda que filtra en tiempo real. También puede apoyarse en el siguiente video tutorial:

Video de referencia: "LazyColumn en Android Jetpack Compose. Lista con texto y botones" https://www.youtube.com/watch?v=C4EkfVFC0cg

Paso 1. Crear un nuevo proyecto.

nombre sugerido para el proyecto: "ListaFiltrable_Compose", Package name: com.example.listafiltrable_compose, Language: Kotlin, Minimum SDK: el que venga por defecto en la plantilla (recuerden usar un emulador o dispositivo con API 34 o superior, según la sección de Recursos a utilizar).

Paso 2. Agregar, debajo del package y antes de la clase MainActivity, todas las importaciones que se usarán en este ejercicio:

Código (importaciones):

```
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
```

```
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material3.Card
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.listafiltrable_compose.ui.theme.ListaFiltrable_ComposeTheme
```

Paso 3. Definir, fuera de cualquier Composable, la data class que representará cada edificio:

Código (data class Edificio):

```
data class Edificio(
    val id: Int,
    val nombre: String,
    val categoria: String,
    val descripcion: String
)
```

Paso 4. Crear el Composable ListaEdificios, con una lista de muestra de al menos 6 edificios (remember) y una variable de estado para el texto de búsqueda. Por ahora dejamos el resto del cuerpo vacío; lo iremos completando en los siguientes pasos:

Código (ListaEdificios – Paso 4):

```
@Composable
fun ListaEdificios() {
    val edificios = remember {
        listOf(
            Edificio(1, "Biblioteca Central", "Servicios", "Préstamo de libros y salas de
estudio"),
            Edificio(2, "Comedor Universitario", "Servicios", "Alimentación para estudiantes"),
            Edificio(3, "Facultad de Ingeniería", "Académico", "Aulas y laboratorios de
ingeniería"),
            Edificio(4, "Facultad de Ciencias", "Académico", "Aulas y laboratorios de ciencias"),
            Edificio(5, "Coliseo Universitario", "Deportes", "Actividades deportivas y eventos"),
            Edificio(6, "Auditorio Central", "Eventos", "Conferencias y ceremonias")
        )
    }
    var texto by remember { mutableStateOf("") }

    // el campo de búsqueda y la lista se agregan en los siguientes pasos
}
```

Paso 5. Encima de la lista, agregar un OutlinedTextField ligado a la variable texto:

Código (ListaEdificios – Paso 5):

```
    var texto by remember { mutableStateOf("") }

    Column(modifier = Modifier.fillMaxSize().padding(16.dp)) {
        OutlinedTextField(
            value = texto,
            onValueChange = { texto = it },
            label = { Text("Buscar edificio") },
            modifier = Modifier.fillMaxWidth()
        )

        Spacer(modifier = Modifier.height(12.dp))

        // la lista filtrada se agrega en el Paso 6
    }
```

Paso 6. Calcular la lista filtrada a partir de texto, recalculándola en cada recomposición:

Código (ListaEdificios – Paso 6):

```
        Spacer(modifier = Modifier.height(12.dp))

        val edificiosFiltrados = edificios.filter {
            it.nombre.contains(texto, ignoreCase = true)
        }

        // el LazyColumn se agrega en el Paso 7
    }
```

Paso 7. Declarar el LazyColumn, usando items() con una key estable para dibujar cada fila a partir de la lista filtrada. Con esto, ListaEdificios queda completo:

Código (ListaEdificios completo):

```
@Composable
fun ListaEdificios() {
    val edificios = remember {
        listOf(
            Edificio(1, "Biblioteca Central", "Servicios", "Préstamo de libros y salas de
estudio"),
            Edificio(2, "Comedor Universitario", "Servicios", "Alimentación para estudiantes"),
            Edificio(3, "Facultad de Ingeniería", "Académico", "Aulas y laboratorios de
ingeniería"),
            Edificio(4, "Facultad de Ciencias", "Académico", "Aulas y laboratorios de ciencias"),
            Edificio(5, "Coliseo Universitario", "Deportes", "Actividades deportivas y eventos"),
            Edificio(6, "Auditorio Central", "Eventos", "Conferencias y ceremonias")
        )
    }
    var texto by remember { mutableStateOf("") }

    Column(modifier = Modifier.fillMaxSize().padding(16.dp)) {
        OutlinedTextField(
            value = texto,
            onValueChange = { texto = it },
            label = { Text("Buscar edificio") },
            modifier = Modifier.fillMaxWidth()
        )

        Spacer(modifier = Modifier.height(12.dp))

        val edificiosFiltrados = edificios.filter {
            it.nombre.contains(texto, ignoreCase = true)
        }
```

```
        LazyColumn(modifier = Modifier.fillMaxSize()) {
            items(edificiosFiltrados, key = { it.id }) { edificio ->
                FilaEdificio(edificio)
            }
        }
    }
}
```

Paso 8. Implementar el Composable FilaEdificio, mostrando dentro de una Card al menos tres datos: el nombre (en negrita), la categoría y la descripción:

Código (FilaEdificio):

```
@Composable
fun FilaEdificio(edificio: Edificio) {
    Card(
        modifier = Modifier
            .fillMaxWidth()
            .padding(vertical = 6.dp)
    ) {
        Column(modifier = Modifier.padding(12.dp)) {
            Text(
                text = edificio.nombre,
                fontWeight = FontWeight.Bold,
                fontSize = 18.sp
            )
            Text(
                text = edificio.categoria,
                fontSize = 14.sp,
                color = Color.Gray
            )
            Text(
                text = edificio.descripcion,
                fontSize = 14.sp
            )
        }
    }
}
```

Paso 9. Dentro de MainActivity, en el bloque setContent, mostrar ListaEdificios:

Código (MainActivity.kt completo):

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            ListaFiltrable_ComposeTheme {
                Surface(modifier = Modifier.fillMaxSize()) {
                    ListaEdificios()
                }
            }
        }
    }
}

@Preview(showBackground = true)
@Composable
fun ListaEdificiosPreview() {
    ListaFiltrable_ComposeTheme {
        ListaEdificios()
    }
}
```

Paso 10. Ejecutar la aplicación en un emulador, escribir distintos textos en el campo de búsqueda y verificar que la lista se actualiza mostrando solo las coincidencias en cada pulsación.

Paso 11. Probar el caso sin coincidencias (un texto que no existe en ningún nombre) y confirmar que la lista queda vacía sin errores.

III. EJERCICIOS/PROBLEMAS PROPUESTOS

Amplía el ejercicio resuelto para mostrar la lista completa de edificaciones históricas del campus:

1. Reemplaza la lista de muestra por al menos 8 edificios reales del campus, cargados desde un archivo de texto plano (similar al patrón de persistencia usado en el Laboratorio 03) en lugar de estar escritos directamente en el código.
2. Agrega a FilaEdificio un ícono o imagen representativa (puede ser un Icon de Material Icons si no cuentas con fotos propias).
3. Permite elegir el criterio de filtro: agrega un segundo campo o un menú desplegable que permita filtrar por categoría además de por nombre.
4. (Reto opcional) Integra esta lista filtrable como reemplazo de EdificiosScreen (Laboratorio 05/06), combinándola con las tarjetas TarjetaLugar o TarjetaCompacta del Laboratorio 06 dentro de cada fila.

Entregable: informe con capturas de la lista completa, del filtrado en acción con al menos dos textos de búsqueda distintos, y una breve descripción de cómo se cargan los datos desde el archivo.

IV. CUESTIONARIO

1. ¿Lograste resolver completamente el Ejercicio/Problema Resuelto por el Docente (lista de edificaciones que se filtra en tiempo real al escribir en el campo de búsqueda)? Adjunta una captura de pantalla del emulador mostrando el resultado como evidencia.
2. En el video de referencia (https://www.youtube.com/watch?v=C4EkfVFC0cg) se aborda el uso de LazyColumn. ¿Qué términos o conceptos nuevos, o que amplían lo visto en la sesión de teoría, se mencionan en el video? Explica brevemente.
3. ¿Por qué es recomendable pasar un parámetro key = { it.id } a la función items() de una LazyColumn? ¿Qué podría ocurrir si no se especifica al filtrar o reordenar la lista?

V. REFERENCIAS Y BIBLIOGRÁFIA RECOMENDADAS:

[1] Android Developers, "Lists and grids". [En línea]. Disponible: https://developer.android.com/develop/ui/compose/lists

[2] "LazyColumn en Android Jetpack Compose. Lista con texto y botones", video. [En línea]. Disponible: https://www.youtube.com/watch?v=C4EkfVFC0cg

TÉCNICAS E INSTRUMENTOS DE EVALUACIÓN

TÉCNICAS:	**INSTRUMENTOS:**
Ejercicios propuestos	Rúbrica de evaluación

CRITERIOS DE EVALUACIÓN

- Cumplimiento de los objetivos y ejercicios propuestos en la guía.
- Correcta aplicación de los conceptos y herramientas del tema desarrollado.
- Funcionamiento correcto de lo implementado (compilación/ejecución sin errores).
- Informe (formato estudiante) completo, con evidencias y cuestionario resuelto.
- Entrega dentro del plazo establecido.
- Se permite el uso de Inteligencia Artificial (IA) como fuente de consulta o apoyo para resolver los ejercicios; sin embargo, el desarrollo completo del laboratorio no puede basarse únicamente en

respuestas generadas por IA. El estudiante debe evidenciar comprensión propia mediante la justificación razonada de sus respuestas, especialmente en el cuestionario.